

datalab 报告

姓名:冉鸿杰希

学号:2025201868

总分	bitAnd	bitXor	samesign	logtwo	byteSwap	reverse	logicalShift	leftBitCount	float_i2f	floatScale2	float64_f2i
37	1	1	2	4	4	3	3	4	4	4	3

test 截图:

```
Luoxi@LAPTOP-CC342QSS:/mnt/d/study/program design/computer frustration/datalab2026$ sudo apt-get update && sudo apt-get install build-essential gcc-multilib
cd "/mnt/d/study/program design/computer frustration/datalab2026"
make
./btest
python3 test.py
[sudo: authenticate] Password:
Hit:1 http://security.ubuntu.com/ubuntu resolute-security InRelease
Hit:2 http://archive.ubuntu.com/ubuntu resolute InRelease
Hit:3 http://archive.ubuntu.com/ubuntu resolute-updates InRelease
Hit:4 http://archive.ubuntu.com/ubuntu resolute-backports InRelease
Reading package lists... Done
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
build-essential is already the newest version (12.12ubuntu2.26.04.2).
gcc-multilib is already the newest version (4:15.2.0-5ubuntu1).
Solving dependencies... Done
0 upgraded, 0 newly installed, 0 to remove and 90 not upgraded.
make: Nothing to be done for 'all'.
1 1 0 bitAnd
1 1 0 bitXor
2 2 0 byteSwap
3 3 0 logicalShift
4 4 0 float_i2f
4 4 0 leftBitCount
4 4 0 floatPower2
4 4 0 floatScale2
3 3 0 reverse
2 2 0 sameSign
4 4 0 logtwo
3 3 0 float64_f2i
rm -f *.o btest fshow ishow *.~ yacctab.py lextab.py result.txt .autograder_result
gcc -O0 -Wall -std=gnu99 -lm -o btest bits.c btest.c decl.c tests.c
gcc -O0 -Wall -std=gnu99 -o fshow fshow.c
gcc -O0 -Wall -std=gnu99 -o ishow ishow.c
Make success.
bitAnd 1/1: PASS
bitXor 1/1: PASS
sameSign 2/2: PASS
logtwo 4/4: PASS
byteSwap 4/4: PASS
reverse 3/3: PASS
logicalShift 3/3: PASS
leftBitCount 4/4: PASS
float_i2f 4/4: PASS
floatScale2 4/4: PASS
float64_f2i 3/3: PASS
floatPower2 4/4: PASS
Total points: 37
```

解题报告

亮点

- 1. **leftBitCount**:把"数前导 1"转化成"位涂抹 + 分治数 1",全程只用右移和加法(减法用补码替换),31 ops,0 和 32 两个边界都不需要特判。
- 2. **float64_f2i**:用"尾数对齐到 bit31 再右移"一步同时完成取整和向零舍入,并让非规格化数、溢出、恰好 -2^{31} 三种边界共用同一套判断,约 15 ops(上限 60)。

bitAnd

```
int bitAnd(int x, int y) {
    return ~(~x | ~y);
}
```

根据德摩根律。两位同时为 1,等价于"不存在某位为 0":两数取反后或运算,1 的位置就是原来至少有一个 0 的位置,再整体取反还原。

bitXor

```
int bitXor(int x, int y) {
    return ~(~x & ~y) & ~(x & y);
}
```

要实现异或操作,就是要有 1 但是不全为 1 就行了。两个条件分别由 $\sim(\sim x \ \& \ \sim y)$ 和 $\sim(x \ \& \ y)$ 给出。代表 $(x \ | \ y)$ 和 $\sim(x \ \& \ y)$

samesign

```
int samesign(int x, int y) {
    if (!x) return !y;
    if (!y) return 0;
    return !((x >> 31) ^ (y >> 31));
}
```

实现同号判断,先注意到 0 非正也非负,单独算一类:x 为 0 时结果取决于 y 是否为 0。两数都非零时,算术右移 31 位把符号位变成符号位,全 0 或全 1,异或后取非即得同号判定。

logtwo

```
int logtwo(int v) {
    int r = 0;
    int t;

    t = v > 0xFFFF;
    v = v >> (t << 4);
    r = r | (t << 4);
    t = v > 0xFF;
    v = v >> (t << 3);
    r = r | (t << 3);
    t = v > 0xF;
    v = v >> (t << 2);
    r = r | (t << 2);
    t = v > 0x3;
    v = v >> (t << 1);
    r = r | (t << 1);
    r = r | (v > 1);
    return r;
}
```

$\log_2(v)$ 本质上就是找最高位 1 的位置。一开始想不通就做不起,后面问了AI才想起来。
二分搜索是最好想的,用r来记录结果,每次判断「v 是否还大于某个 2 的幂阈值」：

步骤	阈值	判断 $t = v >$ 阈值	若 t=1 则右移	r 累加
1	$0xFFFF = 2^{16}-1$	v 是否 $\geq 2^{16}$	16 位	+16
2	$0xFF = 2^8-1$	v 是否 $\geq 2^8$	8 位	+8
3	$0xF = 2^4-1$	v 是否 $\geq 2^4$	4 位	+4
4	$0x3 = 2^2-1$	v 是否 $\geq 2^2$	2 位	+2

步骤	阈值	判断 $t = v > \text{阈值}$	若 $t=1$ 则右移	r 累加
5	1	v 是否 $\geq 2^1$	不移	+1

byteSwap

```
int byteSwap(int x, int n, int m) {
    int n8 = n << 3;
    int m8 = m << 3;
    int bn = (x >> n8) & 0xFF;
    int bm = (x >> m8) & 0xFF;
    int d = bn ^ bm;
    return x ^ ((d << n8) | (d << m8));
}
```

第 k 字节 = $(x \gg 8k) \& 0xFF$, 我一开始不知道有第0字节, 写错了一遍。设两字节差值 d , 则 $x \wedge (d \ll 8n \mid d \ll 8m)$ 让这两个位置的字节各自翻转成对方(一个位置异或 $bn \wedge bm$ 得到 bm , 另一个得到 bn)

$$bn \wedge d = bn \wedge (bn \wedge bm) = (bn \wedge bn) \wedge bm = bm$$
$$bm \wedge d = bm \wedge (bn \wedge bm) = bm \wedge bm \wedge bn = bn$$

这就是不用临时变量交换两个值的老技巧 ($a^=b$; $b^=a$; $a^=b$;) 的变形。

reverse

```
unsigned reverse(unsigned v) {
    v = (v >> 16) | (v << 16);
    v = ((v & 0xFF00FF00) >> 8) | ((v & 0x00FF00FF) << 8);
    v = ((v & 0xF0F0F0F0) >> 4) | ((v & 0x0F0F0F0F) << 4);
    v = ((v & 0xCCCCCCC) >> 2) | ((v & 0x33333333) << 2);
    v = ((v & 0xAAAAAAAA) >> 1) | ((v & 0x55555555) << 1);
    return v;
}
```

蝴蝶分治: 先交换左右 16 位, 再在每个 16 位里交换 8 位.....最后交换相邻奇偶位, 5 轮后整体反转。每轮用一对互补掩码分别取出左移部分和右移部分再拼回。

F: 1111 0: 0000 C: 1100 3: 0011 A: 1010 5: 0101

logicalShift

```
int logicalShift(int x, int n) {
    return (x >> n) & ~(((1 << 31) >> n) << 1);
}
```

题目要求实现逻辑右移, 实际上就是在算术右移的基础上将高位的符号位换为0。而算术右移会把符号位复制进高 n 位, 需要造一个"恰好高 n 位为 1"的掩码清掉它们。 $1 \ll 31$ 只有符号位, 算术右移 n 位后高 $n+1$ 位全 1, 再左移 1 位挤掉多出的一个, 剩下正好 n 个 1, 取反就是保留掩码了。

leftBitCount

```
int leftBitCount(int x) {
    int y = ~x;
    y = y | (y >> 1);
    y = y | (y >> 2);
    y = y | (y >> 4);
    y = y | (y >> 8);
    y = y | (y >> 16);
    y = y + (~(y >> 1) & 0x55555555) + 1);
    y = (y & 0x33333333) + ((y >> 2) & 0x33333333);
    y = (y + (y >> 4)) & 0x0F0F0F0F;
    y = y + (y >> 8);
    y = y + (y >> 16);
    y = y & 0x3F;
    return 32 + (~y + 1);
}
```

题目要求我返回从最高位端开始连续 1 的个数。写这题最麻烦的是不给减法、也不给 if 和比较,要用 $a + (\sim b + 1)$ 代替 $a - b$,我写着写着就会被绕晕,后来发现可以先带带减法的正常逻辑写,全部写顺之后再把减法整体替换掉,这样逻辑就不乱了:

例如 $y = y + (\sim((y >> 1) \& 0x55555555) + 1)$; 就是 $y = y - ((y >> 1) \& 0x55555555)$ 。

我的思路是一条转化链:

数 x 的前导连续1 ——取反——► 数 $y = \sim x$ 的前导连续0 ——涂抹——► 数 1 的个数(popcount) ——32减——► 答案

取反:先 $y = \sim x$,把"数前导 1"变成"数前导 0"。

涂抹:依次 $y = y | (y >> 1)$ 、 $>>2$ 、 $>>4$ 、 $>>8$ 、 $>>16$ 。覆盖宽度每轮翻倍($1+1+2+4+8+16=32$),最高位的 1 就把它以下的所有位都染成 1。这样"最高位 1 在哪"这个位置信息就变成了"1 有多少个"的数量信息:涂抹后 1 的个数是最高位 1 及其**以下**的位数,而要数的前导 0 在它**上面**,个数正好是 $32 - \text{popcount}$ ——这就是最后一步"32 减"的来历。

popcount:分治着数 1:先每 2 位一组(掩码 0x55555555,把每对里高位的值减掉,剩下的正好是每对 1 的个数),再每 4 位、每字节、全字逐级相加,最后 & 0x3F(结果最大 32,6 位够存)。

以 `leftBitCount(0xFFFF0F0F0)` 为例: $\sim x = 0x000F0F0F \rightarrow$ 涂抹后 `0x000FFFFF`(popcount = 20,最高位 1 在 bit19) $\rightarrow$ 前导 0 = $32 - 20 = 12 \checkmark$ 。

边界不用特判: $x = -1$ 时 $\sim x = 0$,popcount 为 0,返回 32; $x \geq 0$ 时 $\sim x$ 符号位为 1,涂抹后全 1,popcount 为 32,返回 0。全程只有右移和加法(减法都是补码替换的),没有对负数左移的风险,也不依赖右移是算术还是逻辑,31 ops(上限 50)。

float_i2f

```
unsigned float_i2f(int x) {
    unsigned ux, sign, frac, drop, half, kept;
    int e;

    if (x == 0) return 0;
    ux = x;
    sign = 0;
    if (ux >> 31) { sign = 0x80000000; ux = -ux; }
    e = 31;
    while (!(ux >> e)) e = e - 1;
    if (e > 23) {
        int d = e - 23;
        kept = ux >> d;
        drop = ux - (kept << d);
        half = 1 << (d - 1);
        kept = kept & 0x7FFFFF;
        if (drop > half) kept = kept + 1;
        else if (drop == half) kept = kept + (kept & 1);
        frac = kept;
    } else {
        frac = (ux << (23 - e)) & 0x7FFFFF;
    }
    return sign + ((e + 127) << 23) + frac;
}
```

流程:0 特判 → 记符号、取绝对值(unsigned 求负 = 模 2^{32} 减法,INT_MIN 也正确)→ while 从 31 往下找最高位 1 得阶码真值 e → 求尾数。

尾数两种情况: $e \leq 23$ 时小数位放得下,左移 $23-e$ 再用 0x7FFFFF 去掉隐含的 1; $e > 23$ 时要丢掉 d 位,需要**向偶数舍入**:丢掉的值超过一半($2^{(d-1)}$)进位,正好一半且保留部分末位为 1 也进位。

一个细节:最后打包用 $+$ 而不是 $|$,因为舍入进位可能让 frac 变成 0x800000,此时加法自动把进位推进阶码字段,结果恰好正确(如 $x=0x7FFFFFFF$:丢 0x7F 超过一半,进位后得 $0x4F000000 = 2^{31}$)。

floatScale2

```
unsigned floatScale2(unsigned uf) {
    unsigned exp = (uf >> 23) & 0xFF;
    if (exp == 255) return uf;
    if (exp == 0)
        return (uf & 0x80000000) | ((uf & 0x7FFFFFFF) << 1);
    return (uf & 0x807FFFFFFF) | ((exp + 1) << 23);
}
```

按阶码字段分三类:255(NaN/无穷)原样返回;0 是非规格化数,值为 $\text{frac} \times 2^{(-149)}$,翻倍即尾数左移 1 位,溢出到 bit23 时该位自然成为新阶码的最低位,恰好是“翻倍后跨入规格化区”的正确编码;其余情况阶码 +1,加到 255 自动变成无穷。

调试记录:最初非规格化分支误用掩码 0x807FFFFFFF(符号+旧尾数), $|$ 无法清掉旧尾数位,导致 floatScale2(0x1) 得 3 而非 2;改为只保留符号位(0x80000000)后通过。

float64_f2i

```
int float64_f2i(unsigned uf1, unsigned uf2) {
    unsigned m;
    int e, result, sign;

    sign = uf2 >> 31;
    e = ((uf2 >> 20) & 0x7FF) - 1023;
    if (e >= 31) return 0x80000000;
    if (e < 0) return 0;
    m = ((uf2 & 0xFFFFF) | 0x100000) << 11;
    m = m | (uf1 >> 21);
    result = m >> (31 - e);
    if (sign) result = -result;
    return result;
}
```

拆位:64 位 double 拆在两个参数里,uf2 是高 32 位:bit31 符号,bit30..20 是 11 位阶码(偏置 1023),尾数高 20 位在 uf2 低 20 位,尾数低 32 位整个是 uf1。

三种边界共用两次比较:

- 阶码字段全 0(非规格化数)时,e = 0 - 1023 是一个很大的负数,和"值| < 1"的情况一起被 $e < 0 \rightarrow$ 返回 0 覆盖(截断即向零舍入);
- $e \geq 31$ 意味着 |值| $\geq 2^{31}$,超出 int 范围,返回 0x80000000。有趣的是 -2^{31} 本身($e=31$,尾数全 0)是唯一能被 int 表示的这种情况,而它在 int 里的位模式恰好就是 0x80000000,所以不需要为它单独开分支。

核心一行: $((uf2 \& 0xFFFFF) | 0x100000) \ll 11 | (uf1 \gg 21)$ 把"隐含的 1 + 尾数高 20 位"放到 bit31..11,再拼上尾数低位的前 11 位——即把 53 位尾数的最高 21 位对齐到机器字顶部。随后 $m \gg (31 - e)$ 一次移位同时完成两件事:取出整数部分、丢弃小数部分(正数右移截断 = 向零舍入)。负数最后统一取反。

小例子: $5.75 = 1.0111 \times 2^2$,uf2 = 0x40170000。e = 2;m = $(0x70000 | 0x100000) \ll 11 = 0xBC000000$; $0xBC000000 \gg 29 = 5$ √(小数部分 .75 被截断)。

约 15 ops(上限 60)。

floatPower2

```
unsigned floatPower2(int x) {
    if (x > 127) return 0x7F800000;
    if (x >= -126) return (x + 127) << 23;
    if (x >= -149) return 1 << (x + 149);
    return 0;
}
```

2^x 的尾数恒为 0,只有阶码在动,按 x 分段即可:[-126,127] 是规格化数,阶码 = x+127 直接移到位;更大溢出返回 +INF;[-149,-127] 落在非规格化区,值 = $\text{frac} \times 2^{-(149)}$,取 $\text{frac} = 2^{(x+149)}$;再小返回 0。